MODULE 3

# Full-Stack Development at Speed

*Renaissance Developer Academy — Detailed Lesson Plan*

Week 3 of 10 | 5 Days | Full-Time Intensive
*Badge Earned: Full-Stack Builder*

Prerequisite: Module 2 — AI-Augmented Developer
DRAFT — March 2026

# Module Overview

Modules 1 and 2 built the mindset and verification skills. Module 3 is where the Renaissance Developer starts to deliver at the speed the role demands. This week, students build a **production-grade full-stack application from scratch in five days** — frontend, API, database, containerization, CI/CD, and cloud deployment — using AI tools at every stage and verifying with the frameworks they developed in Module 2.

The week introduces **Antigravity's Manager View** for multi-agent parallel development: orchestrating 3–5 AI agents working simultaneously on frontend, backend, and infrastructure. This is not vibe coding — it's **directed orchestration**, where the engineer acts as the Creative Director managing a team of AI workers.

**Connection to the JD:** This module maps to Full-Stack Development ("work comfortably in any language, deliver production solutions in days not months"), DevOps & CI/CD ("build complete pipelines end-to-end"), and Cloud Platforms ("design cloud-native solutions, manage infrastructure as code"). The week-long build also exercises Rapid Prototyping & Validation.

## Learning Objectives

1. Ship a complete full-stack application (frontend + API + database) in one week, from architecture to deployed production
2. Orchestrate multi-agent parallel development using Antigravity's Manager View
3. Containerize applications with Docker and deploy cloud-agnostically
4. Build a production CI/CD pipeline with GitHub Actions: matrix builds, caching, Docker publishing, multi-environment deployment
5. Practice database design, schema migrations, and query optimization at speed
6. Apply Module 2 verification skills (Prompt Playbook, AI Code Review Checklist, Bug Log) throughout a sustained build

## Pre-Work (Assigned End of Module 2)

**Due before Day 1. Estimated time: 2–3 hours.**

1. **Stack Selection:** Choose the full-stack framework combination you'll use this week. Recommended combos: (a) React + Node/Express + PostgreSQL, (b) React + Python/FastAPI + PostgreSQL, (c) Vue/Svelte + Go + SQLite. Pick what you want to learn or deepen — AI tools will accelerate the unfamiliar parts.
2. **NotebookLM:** Generate a Module 3 Audio Overview. Upload reference docs for your chosen stack (official docs, tutorials). Generate flashcards for key concepts (REST design, database normalization, Docker fundamentals).
3. **Docker Basics:** If new to Docker, complete Docker's official "Get Started" guide (Parts 1–4). Verify: docker run hello-world works on your machine.

4. **Application Idea:** Come with a clear idea for a full-stack application you want to build. It should solve a real problem and have a defined user. Write a 1-paragraph description and a list of 5–8 user stories.

## Week-at-a-Glance

| Day | Theme | Primary Tools | Key Output |
|---|---|---|---|
| **Day 1** | Architecture, Database & API Foundation | Kiro, Claude Code, GitHub Actions | Spec docs, DB schema, running API with tests |
| **Day 2** | Frontend & Multi-Agent Orchestration | Antigravity Manager View, Claude Code | Connected frontend + API, parallel dev workflow |
| **Day 3** | Containerization, CI/CD & Deployment | Docker, GitHub Actions, cloud deploy | Dockerized app, full CI/CD, staging deploy |
| **Day 4** | Polish, Testing & Production Hardening | All tools, GitHub Actions | Production deploy, test coverage >80%, monitoring |
| **Day 5** | Demo Day, Architecture Review & Calibration | NotebookLM, all tools | Live demo, architecture doc, development map update |

# Day 1: Architecture, Database & API Foundation

**Theme:** Spec-driven architecture before any frontend code — the backend and data model are the foundation that determines everything else

**Goal:** Every student has a documented architecture (via Kiro specs), a working database with schema migrations, and a tested API with at least 5 endpoints — all deployed locally with CI running.

## Schedule

| | | |
|---|---|---|
| **09:00** | **Week 3 Opening: The Full-Stack Sprint** | 30 min |
| | • This week's framing: "Deliver production solutions in days not months" — the JD's most demanding expectation<br>• The week-long project: a complete full-stack application, spec'd, built, tested, containerized, and deployed<br>• Why backend-first: the database schema and API contract determine the frontend's possibilities<br>• Share application ideas from pre-work — quick round-robin, 30 seconds each<br>• Pair up as "accountability partners" for the week — daily check-ins, mutual code review | |

| | | |
|---|---|---|
| **09:30** | **Spec-Driven Architecture with Kiro** | 90 min |
| | • EXERCISE 1: Use Kiro's spec-driven workflow to architect your application<br>• Give Kiro your 1-paragraph description and user stories from pre-work:<br>• — Review the generated requirements.md — are all user stories captured? What's missing?<br>• — Review the generated design.md — does the proposed architecture make sense? What trade-offs?<br>• — Review the generated tasks.md — is the task breakdown realistic for a 4-day build? Re-scope if not<br>• Enhance the specs manually — add:<br>• — Database schema: tables, relationships, indexes, constraints<br>• — API contract: endpoints, request/response shapes, authentication approach<br>• — Non-functional requirements: response time targets, concurrent user expectations, data volume<br>• Commit all spec documents to your repo before writing any code<br>• Key connection: Kiro's spec workflow IS the Precision mindset from the Renaissance Developer framework — communicate with clarity before executing | |

| | | |
|---|---|---|
| **11:00** | **Break** | 15 min |
| | • Questions about spec quality | |

| 11:15 | **Database Design & Migration Lab** | 75 min |
|---|---|---|

- EXERCISE 2: Build your database foundation
- — Design the schema: entities, relationships, and data types informed by your spec documents
- — Write the initial migration (using your stack's migration tool: Prisma, Alembic, Knex, GORM, etc.)
- — Seed the database with realistic test data (at least 50 records per main table)
- — Write 3 complex queries you'll need for the API (joins, aggregations, filters)
- Use Claude Code CLI for the implementation — but verify schema design against your spec
- Common AI pitfalls in database work (apply your Bug Log knowledge):
- — Over-normalization or under-normalization — check: does the schema serve your actual queries?
- — Missing indexes on frequently queried columns
- — No foreign key constraints (AI sometimes "forgets" referential integrity)
- — Schema that doesn't account for soft deletes, timestamps, or versioning

| 12:30 | **Lunch** | 60 min |
|---|---|---|

- Optional: database design office hours

| 13:30 | **API Build Sprint** | 120 min |
|---|---|---|

- EXERCISE 3: Build the API layer
- Using Claude Code CLI (primary) and your Kiro specs (reference):
- — Implement at least 5 CRUD endpoints matching your API contract from the spec
- — Add input validation on every endpoint (never trust client data — Bug Safari lesson)
- — Add proper error handling: consistent error response format, appropriate HTTP status codes
- — Add authentication (JWT or session-based — choose one, implement properly)
- — Write tests for every endpoint: happy path + at least 2 error cases each
- Use your Module 2 AI Code Review Checklist on all generated API code — especially:
- — Security section: is auth implemented correctly? Input validation complete?
- — Performance section: are database queries efficient? Any N+1 risks?
- — Test quality section: do tests actually verify behavior?
- Commit frequently (every 20 min) with descriptive messages
- 14:30 milestone check: "How many endpoints are working? Tests passing?"

| 15:30 | **Break** | 15 min |
|---|---|---|

- Stretch

| 15:45 | **CI/CD Foundation & API Verification** | 45 min |
|---|---|---|
| | • EXERCISE 4: Set up your Week 3 CI/CD pipeline<br>• — Create .github/workflows/ci.yml: lint → test → build on push and PR<br>• — Add database service container (PostgreSQL or SQLite in CI) for integration tests<br>• — Add dependency caching for your stack (node_modules, pip cache, etc.)<br>• — Ensure all API tests pass in CI, not just locally<br>• Push and verify: green pipeline before leaving today | |

| 16:30 | **Day 1 Wrap-Up** | 30 min |
|---|---|---|
| | • Accountability partner check: share progress, swap repo links<br>• End-of-day state: spec docs committed, database with migrations and seed data, 5+ tested API endpoints, green CI pipeline<br>• HOMEWORK: Add 2 more endpoints or improve test coverage. Push before morning.<br>• Preview: tomorrow — frontend build and your first multi-agent orchestration with Antigravity's Manager View | |

***Instructor Notes — Day 1:*** *Day 1 sets the tempo for the week. The Kiro spec exercise is non-negotiable — students who skip it and jump straight to code will produce more fragile applications by Week's end. If a student's spec is thin, coach them to expand it before moving to database work. The 120-minute API sprint is long by design — this is the first sustained build session where Module 2's verification skills need to be applied in flow. Watch for students who turn off their Verification Lens when under time pressure — that's exactly the habit this week aims to break. The accountability partner pairing should last all week; check that partners are actually reviewing each other's code.*

# Day 2: Frontend & Multi-Agent Orchestration

**Theme:** Build the frontend layer while learning to orchestrate multiple AI agents working in parallel

**Goal:** Every student has a connected frontend consuming their API, has used Antigravity's Manager View to coordinate parallel development, and understands when multi-agent orchestration helps vs. hinders.

## Schedule

| | | |
|---|---|---|
| **09:00** | **Multi-Agent Orchestration: Theory & Practice (Concept Session)** | 45 min |
| | • From single-agent to multi-agent: why orchestration matters for full-stack development<br>• Antigravity's Manager View: the mission-control interface for parallel AI development<br>• — Agent Workspaces: each agent operates in its own isolated context<br>• — Artifacts: agents produce verifiable outputs (code, tests, screenshots, plans) not just text<br>• — Feedback: you comment on artifacts like reviewing a Google Doc — the agent iterates<br>• Three orchestration patterns:<br>• — Pattern 1 — Layer Split: Agent A does backend, Agent B does frontend, Agent C does tests<br>• — Pattern 2 — Feature Split: Agent A builds auth flow, Agent B builds dashboard, Agent C builds data export<br>• — Pattern 3 — Role Split: Agent A writes code, Agent B reviews code, Agent C writes documentation<br>• When multi-agent helps: parallelizable work with clear boundaries<br>• When multi-agent hurts: tightly coupled code where agents step on each other's changes<br>• Key principle: the engineer's job shifts from writing code to defining boundaries, reviewing artifacts, and resolving conflicts | |

| | | |
|---|---|---|
| **09:45** | **Multi-Agent Frontend Build Sprint** | 90 min |
| | • EXERCISE 5: Build the frontend using Antigravity's Manager View<br>• Set up 2–3 parallel agents:<br>• — Agent 1: Build the core page layouts and navigation (shell, routing, responsive structure)<br>• — Agent 2: Build the data-connected components (forms, tables, dashboards that consume your API)<br>• — (Optional) Agent 3: Build the authentication UI (login, register, protected routes)<br>• Your role as orchestrator: | |

| | |
|---|---|
| | • — Define clear boundaries: which files/directories does each agent own?<br>• — Review artifacts as they're produced — leave feedback, approve or request changes<br>• — Resolve conflicts: when agents produce incompatible code, you decide the integration approach<br>• — Merge incrementally: don't let agents work for 2 hours without reviewing<br>• Apply the Verification Lens to each agent's output — multi-agent amplifies speed AND the risk of subtle bugs<br>• Document your orchestration experience: what worked, what broke, how you resolved it |

| 11:15 | **Break** | 15 min |
|---|---|---|
| | • Stretch, resolve any agent conflicts | |

| 11:30 | **Frontend-API Integration Lab** | 60 min |
|---|---|---|
| | • EXERCISE 6: Connect frontend to backend<br>• — Wire up API calls: implement data fetching for all main views<br>• — Error handling on the frontend: loading states, error states, empty states<br>• — Form validation: client-side validation that mirrors server-side validation (defense in depth)<br>• — Authentication flow: login → store token → attach to API calls → handle expiration<br>• Common AI integration pitfalls to watch for:<br>• — Hardcoded API URLs (should use environment variables)<br>• — Missing CORS configuration (backend doesn't accept frontend origin)<br>• — Token stored in localStorage without XSS protection<br>• — No error boundaries — one failed API call crashes the entire app<br>• Test the full flow: create an account, log in, perform all CRUD operations, log out | |

| 12:30 | **Lunch** | 60 min |
|---|---|---|
| | • Optional: integration debugging office hours | |

| 13:30 | **Feature Build Sprint** | 120 min |
|---|---|---|
| | • EXERCISE 7: Feature expansion — build the features that make your app useful<br>• Now that the skeleton works (frontend + API + database + auth), add real features:<br>• — Pick the 3 highest-priority user stories from your Kiro spec<br>• — Use a mix of tools: Claude Code for backend logic, Antigravity for frontend components<br>• — For each feature: implement → write tests → verify with checklist → commit → push<br>• Accountability partner mid-sprint check (14:30): swap screens, quick 5-min review of each other's progress | |

| | |
|---|---|
| | • — “Is the code clean? Are tests meaningful? Does it actually work end-to-end?”<br>• Goal by end of sprint: your app has 3+ working features beyond basic CRUD<br>• Push a PR with all changes — accountability partner reviews before merge |

| 15:30 | **Break** | 15 min |
|---|---|---|
| | • Stretch | |

| 15:45 | **Frontend Testing & CI Enhancement** | 45 min |
|---|---|---|
| | • EXERCISE 8: Add frontend tests and update CI<br>• — Component tests: test 3–5 key components render correctly with mock data<br>• — Integration tests: test at least 1 complete user flow (e.g., login → create item → verify list)<br>• — Update CI pipeline: add frontend lint, test, and build steps alongside existing backend CI<br>• — (Advanced) Add matrix build: test frontend across Node 18/20, backend across your language versions<br>• Verify: full CI pipeline runs lint + test + build for both frontend and backend on every push | |

| 16:30 | **Day 2 Wrap-Up** | 30 min |
|---|---|---|
| | • Accountability partner check: full-stack app running locally with frontend + API + DB + auth<br>• Multi-agent retrospective: quick share — who loved Manager View? Who found it chaotic? Why?<br>• HOMEWORK: Fix any failing tests, polish the UI, ensure CI is green<br>• Preview: tomorrow we containerize, deploy, and build production-grade CI/CD | |

***Instructor Notes — Day 2:*** *The multi-agent orchestration session is Module 3’s signature innovation. Some students will thrive with Manager View; others will find it frustrating when agents produce conflicting code. Both reactions are valuable learning. Coach frustrated students toward better boundary definition rather than abandoning multi-agent entirely — the skill of “defining what each worker owns” is a management skill, not just a technical one. The 120-minute feature sprint is intentionally long — this is where students experience the difference between “building a demo” and “building a product.” The accountability partner check at 14:30 prevents students from going dark for 2 hours and producing unreviewed code.*

# Day 3: Containerization, CI/CD & Deployment

**Theme:** Transform your local application into a containerized, deployable product with a production-grade CI/CD pipeline

**Goal:** Every student has a Dockerized application, a multi-stage CI/CD pipeline publishing to a container registry, and a working deployment on their chosen cloud platform.

## Schedule

| 09:00 | **Containerization Fundamentals (Concept + Live Coding)** | 60 min |
|---|---|---|
| | • Docker mental model: images as blueprints, containers as running instances, layers as caching units<br>• Live-code a multi-stage Dockerfile from scratch (not copy-paste):<br>• — Stage 1 (Build): install dependencies, compile/bundle the application<br>• — Stage 2 (Production): copy only the build artifacts, minimal base image, non-root user<br>• — Why multi-stage: smaller images, faster deploys, reduced attack surface<br>• Docker Compose for local development: app + database + optional services in one command<br>• — Live-code a docker-compose.yml: web service, db service, volumes, networks, health checks<br>• — Environment variable management: .env files, docker-compose overrides<br>• Common AI Docker pitfalls (from the Bug Log):<br>• — Running as root (security risk), missing .dockerignore (bloated images)<br>• — Not pinning base image versions (builds break when latest changes)<br>• — Copying node_modules into image instead of installing in build stage | |

| 10:00 | **Hands-On: Containerize Your Application** | 90 min |
|---|---|---|
| | • EXERCISE 9: Dockerize your full-stack application<br>• — Write a Dockerfile for the backend (multi-stage build)<br>• — Write a Dockerfile for the frontend (build stage + nginx serve stage)<br>• — Write docker-compose.yml: backend + frontend + database + volumes<br>• — Add .dockerignore for both services<br>• — Verify: docker compose up starts the entire application from scratch<br>• — Verify: docker compose down && docker compose up still works (data persists in volumes)<br>• Use Claude Code CLI to help write Dockerfiles — but apply the verification checklist:<br>• — Is the base image pinned? Is multi-stage used? Is it running as non-root?<br>• — Is .dockerignore present? Is the build cache efficient?<br>• — Are environment variables configurable, not hardcoded?<br>• Commit all Docker files and update your README with Docker setup instructions | |

| 11:30 | **Break** | 15 min |
|---|---|---|
| | • Troubleshoot Docker issues | |

| 11:45 | **Production CI/CD Pipeline Build** | 75 min |
|---|---|---|
| | • EXERCISE 10: Build a production-grade GitHub Actions pipeline<br>• Extend your existing CI pipeline with deployment capabilities:<br>• — Stage 1 (Test): lint + unit tests + integration tests (from existing pipeline)<br>• — Stage 2 (Build): Docker image build for backend and frontend<br>• — Stage 3 (Publish): push Docker images to GitHub Container Registry (ghcr.io)<br>• — Stage 4 (Deploy Staging): deploy to staging environment with health check verification<br>• — Stage 5 (Deploy Production): manual approval gate → deploy to production<br>• Advanced features to implement:<br>• — GitHub Environments: create “staging” and “production” environments with different secrets<br>• — Approval gates: production deployment requires manual approval<br>• — Secrets management: database URLs, API keys stored as GitHub Secrets, never in code<br>• — Build caching: cache Docker layers across CI runs for faster builds<br>• — Semantic version tags: images tagged with git SHA and semver | |

| 13:00 | **Lunch** | 60 min |
|---|---|---|
| | • Optional: cloud provider selection discussion | |

| 14:00 | **Cloud Deployment Lab** | 90 min |
|---|---|---|
| | • EXERCISE 11: Deploy to your chosen cloud platform<br>• No specific cloud is mandated — choose based on your learning goals:<br>• — Option A: Railway/Render/Fly.io — simplest, great for fast deployment, limited customization<br>• — Option B: DigitalOcean App Platform/Droplets — balanced control and simplicity<br>• — Option C: AWS (ECS/Fargate or EC2) — enterprise-grade, most to learn<br>• — Option D: GCP (Cloud Run) or Azure (Container Instances) — serverless containers<br>• Regardless of platform, your deployment must:<br>• — Use Docker containers (not platform-specific build systems)<br>• — Have separate staging and production environments (or at least a staging concept)<br>• — Be triggered by your GitHub Actions pipeline (not manual deployment)<br>• — Have a health check endpoint that the pipeline verifies after deployment<br>• — Have environment-specific configuration (staging vs. production database, API URLs) | |

| | |
|---|---|
| | • Document your deployment architecture: a simple diagram showing CI → Registry → Cloud → User<br>• Pair troubleshooting: deployment is where most things break — help each other |

| 15:30 | **Break** | 15 min |
|---|---|---|
| | • Stretch | |

| 15:45 | **Deployment Verification & Pipeline Testing** | 45 min |
|---|---|---|
| | • EXERCISE 12: Verify the full pipeline end-to-end<br>• — Make a code change → push → watch CI run → verify Docker build → verify staging deployment<br>• — Test your staging environment: run through the core user flow<br>• — (If using approval gates) approve production deployment and verify production<br>• — Break something intentionally: push a failing test → verify CI catches it → pipeline stops<br>• — Update your README with deployment badges, staging URL, and production URL<br>• End-of-day state: full pipeline from push to deployed production, verified end-to-end | |

| 16:30 | **Day 3 Wrap-Up** | 30 min |
|---|---|---|
| | • Accountability partner check: swap deployment URLs, test each other's staging environments<br>• Quick share: biggest surprise or frustration with deployment today<br>• HOMEWORK: Fix any deployment issues. Ensure both staging and production are healthy.<br>• Preview: tomorrow — polish, testing, production hardening, and performance | |

***Instructor Notes — Day 3:*** *Day 3 is the most technically dense day of the week and the module where the most students hit unexpected walls. Docker and deployment are where "it works on my machine" dies. Have troubleshooting guides ready for: Docker permission errors on Linux, Docker Desktop memory limits on Mac, port conflicts, database connection strings in containers, and CORS issues in deployed environments. The CI/CD pipeline build (Exercise 10) is ambitious — some students may not finish the full 5-stage pipeline today. That's OK; the priority order is: Test → Build → Publish → Deploy Staging. Production approval gates are a nice-to-have. Encourage students choosing AWS or GCP to pair up — cloud provider debugging is much faster with two sets of eyes.*

# Day 4: Polish, Testing & Production Hardening

**Theme:** Transform a working application into a production-worthy product through comprehensive testing, performance optimization, and operational readiness

**Goal:** Every student's application has >80% test coverage, handles errors gracefully, performs well under load, and has monitoring/logging in place.

## Schedule

| | | |
|---|---|---|
| **09:00** | **Production Readiness: What "Done" Looks Like (Concept Session)** | 30 min |

- The gap between "it works" and "it's production-ready":
- — Error handling: what happens when things fail? Does the user see a stack trace or a friendly message?
- — Performance: does it respond in <200ms? What happens at 100 concurrent users?
- — Observability: if something breaks at 3 AM, can you figure out what happened from the logs?
- — Security: are secrets secure? Is input validated? Are dependencies audited?
- Connection to the JD's Reliability responsibility: "Build resilient systems, establish SLOs/SLIs"
- Today's mantra: "Make it work (done). Make it right (today's morning). Make it observable (today's afternoon)."

| | | |
|---|---|---|
| **09:30** | **Testing Intensive** | 90 min |

- EXERCISE 13: Achieve >80% test coverage
- Audit your current test suite: use a coverage tool (nyc/istanbul, pytest-cov, go test -cover)
- — Identify untested code paths, especially error handlers and edge cases
- — Use Claude Code CLI to generate tests for uncovered paths — but verify using the Bug Safari checklist:
- — Do tests actually assert behavior or just execute code?
- — Are tests independent and deterministic?
- — Do tests cover the edge cases from your Module 2 Bug Log?
- Add integration tests for the 3 most critical user flows:
- — User registration + login + first action
- — Core CRUD flow (create, read, update, delete) end-to-end
- — Error flow: what happens when the database is unavailable? When the API returns 500?
- Update CI pipeline: add coverage threshold — block merge if coverage drops below 80%
- Push and verify: CI pipeline runs tests with coverage reporting

| 11:00 | Break | 15 min |
|---|---|---|
| | • Stretch | |

| 11:15 | Error Handling & Resilience Workshop | 75 min |
|---|---|---|
| | • EXERCISE 14: Production-harden your application<br>• Error handling audit:<br>• — Backend: consistent error response format across all endpoints, structured logging for all errors<br>• — Frontend: error boundaries around major sections, user-friendly error messages, retry logic for transient failures<br>• — Database: connection pooling, timeout handling, retry on connection loss<br>• Security hardening:<br>• — Run npm audit / pip-audit / govulncheck and fix critical vulnerabilities<br>• — Add security headers (CORS, CSP, HSTS) to your API responses<br>• — Verify: no secrets in code, all env vars documented in .env.example<br>• — Add rate limiting to authentication endpoints<br>• Update CI: add security scanning step (Bandit/Semgrep for Python, npm audit for Node) | |

| 12:30 | Lunch | 60 min |
|---|---|---|
| | • Optional: performance tips sharing | |

| 13:30 | Performance & Observability Lab | 90 min |
|---|---|---|
| | • EXERCISE 15: Add observability and measure performance<br>• Logging:<br>• — Structured logging (JSON format): request ID, user ID, action, duration, status<br>• — Log levels: ERROR for failures, WARN for degraded, INFO for business events, DEBUG for development<br>• — No sensitive data in logs (PII, passwords, tokens)<br>• Health check endpoint (if not already present):<br>• — /health returns: app version, uptime, database connectivity, dependency status<br>• — Used by CI/CD pipeline for deployment verification<br>• Performance baseline:<br>• — Use a simple load test tool (hey, wrk, or k6) to measure response times under load<br>• — Target: p50 < 100ms, p95 < 500ms, p99 < 1s for main endpoints<br>• — Identify and fix the slowest endpoint (usually a missing index or N+1 query)<br>• — Document performance results in your architecture document<br>• (Advanced) Add basic monitoring:<br>• — Request duration metrics, error rate tracking, database query timing<br>• — Options: simple middleware logging, Prometheus metrics, or a hosted service (free tiers) | |

| 15:00 | **Break** | 15 min |
|---|---|---|
| | • Stretch | |

| 15:15 | **Architecture Documentation** | 45 min |
|---|---|---|
| | • EXERCISE 16: Write your architecture walkthrough document<br>• This goes in your repo wiki or a ARCHITECTURE.md file:<br>• — System overview: high-level architecture diagram (frontend → API → database)<br>• — Technology choices: what you picked and why (your first ADR — Architecture Decision Record)<br>• — Data model: ER diagram or schema description<br>• — API contract: endpoint summary with auth requirements<br>• — Deployment architecture: CI/CD pipeline → container registry → cloud → user<br>• — Performance baseline: load test results and key metrics<br>• — Known limitations and future improvements<br>• Use NotebookLM to help organize: upload your spec docs and code, ask it to help draft sections | |

| 16:00 | **Day 4 Wrap-Up & Demo Prep** | 30 min |
|---|---|---|
| | • Accountability partner final review: test each other's production deployments, review architecture docs<br>• Demo prep guidance: tomorrow's demo is 5 minutes — structure it now:<br>• — Problem statement (30 sec), live demo (2 min), architecture (1 min), AI tool usage + verification story (1 min), one learning (30 sec)<br>• HOMEWORK: Final polish. Ensure production is stable, README is complete, architecture doc is committed.<br>• HOMEWORK: Practice your 5-minute demo once — time yourself. | |

***Instructor Notes — Day 4:*** *Day 4 is the "make it right" day and it separates students aiming for professional quality from those who stop at "it works." The testing intensive often reveals that AI-generated tests have poor assertion quality (the Bug Safari lesson paying off). Push students to write meaningful assertions. The performance lab is eye-opening — many students have never load-tested anything. Even a simple "hey -n 1000 -c 50" against their API reveals surprising bottlenecks. The architecture document is the week's written deliverable that demonstrates the Precision mindset. Students who struggle with writing should use NotebookLM's Learning Guide mode to help structure their thoughts before writing.*

# Day 5: Demo Day, Architecture Review & Calibration

**Theme:** Present your work, receive structured feedback, reflect on the week, and calibrate for the second half of the program

**Goal:** Every student has delivered a live demo, received peer and instructor feedback, updated their Personal Development Map, and is prepared for Module 4's shift to system design and architecture.

## Schedule

| | | |
|---|---|---|
| **09:00** | **Demo Day: Full-Stack Showcase** | 120 min |
| | • Every student presents: 5 minutes each, strictly timed<br>• Demo format (practice the Precision mindset):<br>• — Problem & user (30 sec): who is this for and what problem does it solve?<br>• — Live demo (2 min): show the deployed production app working end-to-end<br>• — Architecture (1 min): show the architecture diagram, explain key decisions<br>• — AI + verification story (1 min): how you used AI tools, what you caught with your checklist, one Bug Log addition from this week<br>• — One key learning (30 sec): what would you do differently?<br>• After each demo: 2 minutes of structured feedback from the audience:<br>• — One "star" (what was impressive) and one "wish" (what could be better)<br>• Peer voting: "Best architecture", "Best user experience", "Most ambitious scope", "Best AI integration" | |
| **11:00** | **Break** | 15 min |
| | • Celebrate — you just built and deployed a full-stack app in 4 days | |
| **11:15** | **Architecture Peer Review** | 60 min |
| | • EXERCISE 17: Cross-review architecture documents<br>• Each student reviews 2 peers' architecture documents and deployed applications:<br>• — Is the architecture document clear enough for a new developer to understand the system?<br>• — Do the stated technology choices have clear rationale?<br>• — Try to break the deployed app: edge cases, invalid input, unexpected navigation<br>• — Leave structured feedback as GitHub Issues on their repo (labeled "architecture-review")<br>• This previews Module 4's deeper dive into Architecture Decision Records and system design review | |
| **12:15** | **Lunch** | 60 min |

| | |
|---|---|
| | • Informal — discuss surprises from the architecture reviews |

| 13:15 | **Week 3 Retrospective & Development Map Update** | 60 min |
|---|---|---|
| | • Sailboat retrospective (recurring format):<br>• — Wind: What propelled you this week? (often: AI tools making full-stack achievable in 4 days)<br>• — Anchor: What held you back? (often: Docker/deployment complexity, scope management)<br>• — Rocks: What risks lie ahead in Modules 4–10?<br>• — Island: What are you most excited about for the remaining 7 weeks?<br>• EXERCISE 18: Personal Development Map v3 update<br>• — Re-rate on five mindsets: how did Systems thinking grow with full-stack architecture?<br>• — Competency layers: Module 3 targeted Layer 1 (Technical Foundation) and Layer 2 (Architect)<br>• — New gaps discovered? (Common: students realize deployment/DevOps is deeper than expected)<br>• — Mid-program check: are you on track for your 10-week intention?<br>• Instructor 1:1s continue through the afternoon | |

| 14:15 | **Impossible Stuff Day: Self-Directed Exploration** | 105 min |
|---|---|---|
| | • Week 3 Impossible Stuff Day suggestions:<br>• — Add real-time features to your app (WebSockets, Server-Sent Events)<br>• — Add a mobile-responsive PWA configuration (service worker, manifest.json)<br>• — Experiment with a completely different stack: rebuild one feature in Go, Rust, or Elixir with AI<br>• — Write infrastructure-as-code (Terraform, Pulumi) for your deployment<br>• — Explore Kiro’s Agent Hooks: set up auto-test and auto-doc updates on file save<br>• — Read ahead: skim “Designing Data-Intensive Applications” Chapter 1 for Module 4<br>• — Blog post: “Building a full-stack app in 4 days with AI — what’s possible and what’s not”<br>• Share explorations in last 15 minutes | |

| 16:00 | **Badge Ceremony & Week 3 Close** | 30 min |
|---|---|---|
| | • Badge ceremony: Full-Stack Builder badge for students who completed all deliverables<br>• Achievements: best demo, best architecture doc, most creative use of multi-agent orchestration<br>• Read the Developer Manifesto together (recurring ritual)<br>• Midpoint reflection: you’re 30% through the program. You’ve built the mindset (M1), the verification skills (M2), and the building speed (M3). The next three weeks go deeper: system design, data & reliability, and rapid prototyping. | |

| | |
|---|---|
| | • Module 4 preview: System Design & Architecture with Kiro, NanoClaw security analysis, and Architecture Decision Records |

***Instructor Notes — Day 5:*** *Demo Day is a highlight of the program. Strictly enforce 5-minute time limits — this teaches the Precision communication skill. The "star and wish" feedback format keeps it constructive. The architecture peer review exercise previews Module 4's system design focus and often reveals that students whose applications work well may have documentation gaps. The midpoint reflection is important — three weeks in, some students feel energized and some feel overwhelmed. The 1:1 check-ins should identify anyone at risk of burning out. Remind students that the program is designed to be intense but sustainable — Impossible Stuff Day exists precisely to give processing time.*

# Appendix A: Exercise Summary

| # | Day | Exercise | Type |
|---|---|---|---|
| 1 | 1 | Kiro spec-driven architecture: requirements, design, tasks for full-stack app | Architecture |
| 2 | 1 | Database design: schema, migrations, seed data, complex queries | Build |
| 3 | 1 | API build sprint: 5+ endpoints with validation, auth, error handling, tests | Build |
| 4 | 1 | CI/CD foundation: lint → test → build pipeline with database service | DevOps |
| 5 | 2 | Multi-agent frontend build with Antigravity Manager View (2–3 parallel agents) | Orchestration |
| 6 | 2 | Frontend-API integration: data fetching, auth flow, error handling | Integration |
| 7 | 2 | Feature build sprint: 3+ user stories implemented end-to-end | Build |
| 8 | 2 | Frontend testing and CI pipeline enhancement | Testing |
| 9 | 3 | Dockerize: multi-stage Dockerfiles, docker-compose, .dockerignore | DevOps |
| 10 | 3 | Production CI/CD: Docker build → publish → staging deploy → prod deploy | DevOps |
| 11 | 3 | Cloud deployment to chosen platform (cloud-agnostic, container-based) | Deployment |
| 12 | 3 | End-to-end pipeline verification and intentional failure testing | Verification |
| 13 | 4 | Testing intensive: achieve >80% coverage, integration tests for 3 critical flows | Testing |
| 14 | 4 | Production hardening: error handling, security audit, rate limiting | Security |
| 15 | 4 | Observability: structured logging, health checks, performance baseline | Operations |
| 16 | 4 | Architecture documentation: overview, choices, data model, deployment, metrics | Documentation |
| 17 | 5 | Architecture peer review: review 2 peers' apps and docs, file issues | Collaborative |
| 18 | 5 | Personal Development Map v3 update | Reflection |

# Appendix B: Deliverables Checklist

Complete all items to earn the Full-Stack Builder badge:

1. **Spec Documents** — Kiro-generated requirements.md, design.md, and tasks.md, enhanced with database schema and API contract, committed before Day 1 coding began
2. **Full-Stack Application** — Working app with frontend + API + database + authentication, 3+ features beyond basic CRUD, deployed to production
3. **Dockerized Deployment** — Multi-stage Dockerfiles for backend and frontend, docker-compose.yml, .dockerignore, verified with docker compose up from clean state
4. **CI/CD Pipeline** — GitHub Actions: lint → test → build → Docker publish → deploy staging (production deploy optional), with build caching and coverage threshold
5. **Test Suite** — >80% code coverage, integration tests for 3 critical user flows, coverage reporting in CI, all tests passing
6. **Architecture Document** — System overview, technology choices with rationale (ADR format), data model, API contract, deployment diagram, performance baseline, known limitations
7. **Multi-Agent Orchestration Log** — Documentation of Antigravity Manager View experience: agent boundaries, conflict resolution, what worked vs. what didn't
8. **Personal Development Map v3** — Updated self-assessment with midpoint reflection and evidence from Module 3
9. **Peer Contributions** — Architecture reviews of 2 peers' applications (filed as GitHub Issues), accountability partner code reviews throughout the week

---

*End of Module 3 Lesson Plan*